# Graph Algorithms
# Matchings in Graphs

George Mertzios
george.mertzios@durham.ac.uk

Algorithms and Complexity

Lectures 8-9

# Outline

**Learning outcomes:**

1. to be able to explain the MATCHING problem in Graphs
2. to be able to solve it for bipartite graphs.
3. to increase understanding of basic graph algorithms.

# Problem Definition

- Let $G = (V, E)$ be an undirected graph
- When modeling a real-world application, usually:
  - vertices $v \in V$ represent some interesting objects, e.g. cities on a map, antennas that need a frequency ("color"), jobs to be executed, machines executing jobs, ...
  - edges $e \in E$ represent relations between pairs of vertices, e.g. roads between cities, frequency conflicts between transmitters, compatibilities of machines and jobs, ...
- However often the edges become the interesting objects to study:
  - an edge unites or matches two vertices
- A matching $M \subseteq E$ in $G$:
  - an independent (non-conflicting) set of edges, i.e.
  - a set $M$ of edges, where no two edges of $M$ share a common vertex

# It takes two to tango

- Many assignment problems can be formulated as finding a special matching in a bipartite graph $G = (X \cup Y, E)$:
  - vertices in $X$ are agents; vertices in $Y$ are projects to be assigned to agents; each agent can do specific projects
  - a set $X$ of boys wants to dance with a set $Y$ of girls, but only specific pairs of $x \in X$ and $y \in Y$ are "compatible"
  - a set $X$ of tasks needs to be executed by a set $Y$ of machines, but specific machines can only do specific jobs
- Similar situation also in a general (non-bipartite) graph:
  - the roommates problem
  - every vertex $v \in V$ is a student, every edge $e \in E$ means that two students $v_1, v_2$ are "compatible"
  - matching: assigning only compatible student pairs of students into rooms

# Large Matchings

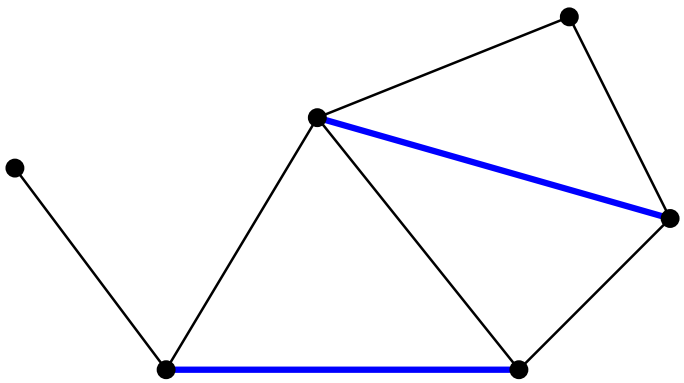
- Trivial matchings: an empty set of edges, a single edge
- We are interested in matchings that are large in cardinality
  - many projects done, many dancing pairs, many roommates
- A matching $M$ is called:
  - maximal: if $M$ is not a proper subset of any other matching $M'$ in $G$
  - maximum: if no other matching $M'$ in $G$ contains a strictly greater number of edges
  - perfect: if every vertex of $G$ is an endpoint of an edge in $M$
- The matching number $\nu_G$ of $G$ is the size of a maximum matching in $G$

| MATCHING | |
|---|---|
| *Instance:* | a graph $G$. |
| *Task:* | determine $\nu_G$ |

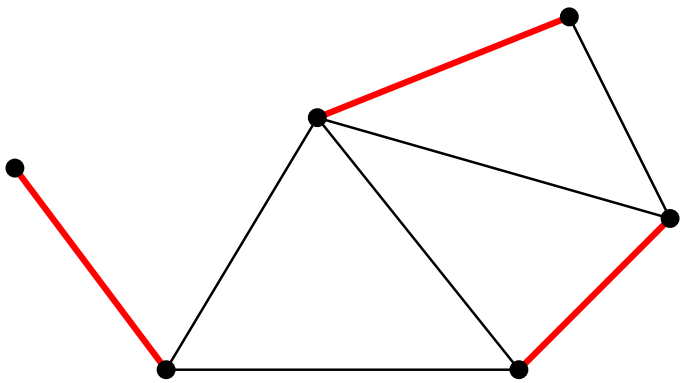
# Large Matchings

Example:

maximal matching:



maximum matching:

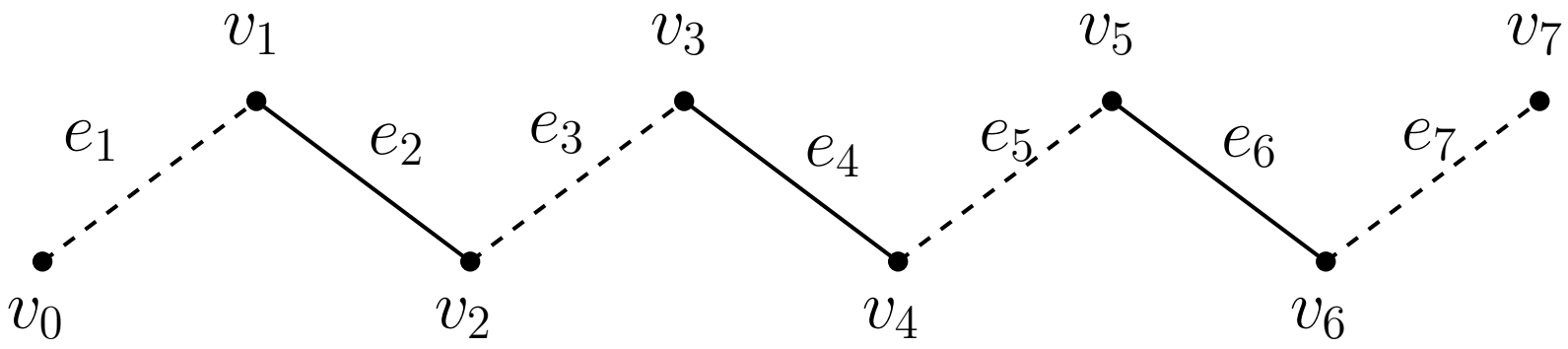


- Every maximum matching is also maximal
  - Note: trivial way to find maximal matchings! (Why?)
- A maximal matching is not necessarily maximum
- A perfect matching is always maximum

# Alternating Paths and Cycles

- Let $G = (V, E)$ be a graph and $M$ be a matching in $G$.
- Let $P = (v_0, v_1, \ldots, v_k)$ be a simple path with edges $e_1 = v_0 v_1, e_2 = v_1 v_2, \ldots, e_k = v_{k-1} v_k$. Then:
- $P$ is an $M$-alternating path if the edges $e_1, e_2, \ldots, e_k$ are alternately in $M$ and in $E \setminus M$.

$$P = (v_0, v_1, v_2, \ldots, v_k) \qquad M = \{e_2, e_4, e_6\}$$




- $P$ is an $M$-augmenting path $P$ is $M$-alternating and neither $v_0$ nor $v_k$ is covered by $M$.

$\Rightarrow$ $M = \{e_2, e_4, \ldots, e_{k-1}\}$

$\Rightarrow$ $k$ is odd in an augmenting path

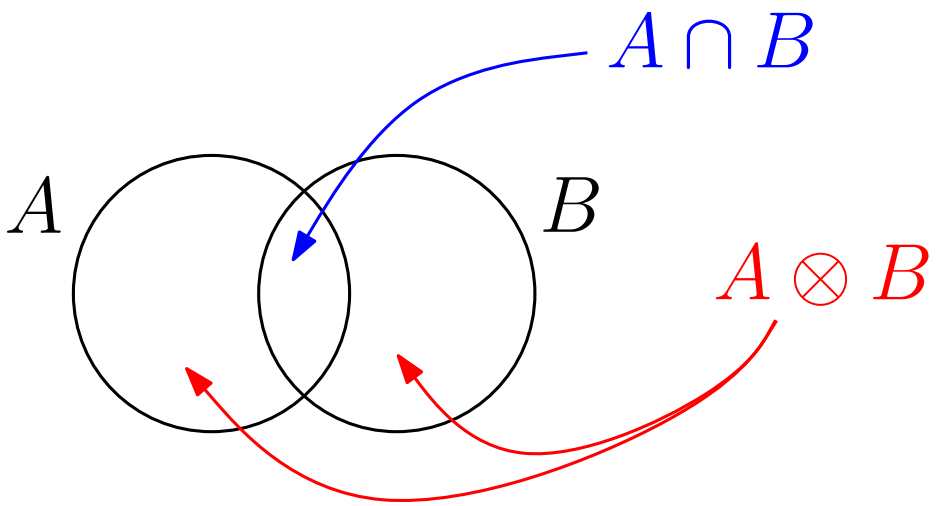
# Alternating Paths and Cycles

- Similarly: a cycle $C$ in $G$ is an $M$-alternating cycle if its edges alternately belong to $M$ and $E \setminus M$.
- A vertex $u$ is matched by $M$ if it is the end-vertex of an edge in $M$; otherwise $u$ is unmatched by $M$.
- For two sets $A$ and $B$, the symmetric difference is the set

$$A \otimes B = (A \setminus B) \cup (B \setminus A).$$

- Example: $S_1 = \{4, 5, 7, 8 4, 5, 7, 8\}$ and $S_2 = \{2, 4, 8, 9 2, 4, 8, 9\}$

$\Rightarrow$ $S_1 \otimes S_2 = \{2, 5, 7, 9\}$



# Alternating Paths and Cycles

A useful lemma:

## Lemma

*Let $G$ be a graph with a matching $M$ and an $M$-alternating path $P = (v_0, v_1, \ldots, v_k)$. If each of the endpoints $v_0, v_k$ of $P$ is:*

- *either unmatched by $M$,*
- *or matched by $M \cap P$,*

*then $M \otimes P$ is another matching in $G$.*

## Corollary

*If $P$ is an $M$-augmenting path, then in this new matching $M \otimes P$:*

- $|M \otimes P| = |M| + 1$,
- *all the matched nodes in $M$ remain matched,*
- *two additional vertices $v_0, v_k$ become matched in $M \otimes P$.*

$\Rightarrow$ *augmentation of $M$: replacing $M$ with $M \otimes P$*

# Augmenting Paths

- Let $G$ be a graph with a matching $M$ and an $M$-augmenting path $P$.
- Then $|M \otimes P| = |M| + 1$, i .e. the matching $M \otimes P$ has more edges than $M$.

$\Rightarrow$ $M$ is not a maximum matching.

- We now prove that the converse holds as well:

  if there exists no $M$-augmenting path $P$, then $M$ is a maximum matching.

# Augmenting Paths

### Lemma (auxiliary)

*Let $G = (V, E)$ be an undirected graph, and let $M$ and $M^*$ be two matchings in $G$. Then the subgraph $H = (V, M \otimes M^*)$ of $G$ is the disjoint union of*

- *isolated vertices*
- *alternating cycles with respect to both $M$ and $M^*$*
- *alternating paths with respect to both $M$ and $M^*$*

# Augmenting Paths

## Lemma

*Let $G = (V, E)$ be a graph. The following are equivalent:*

1. *$M$ is a maximum matching*
2. *$G$ has no $M$-augmenting path*

## Proof (we already proved (1) $\Rightarrow$ (2)).

(2) $\Rightarrow$ (1): Suppose $M$ is not a maximum matching.
Let $M'$ be a maximum matching.
From auxiliary Lemma: the subgraph $H = (V, M \otimes M')$ contains only alternating paths and cycles (with respect to $M$ and $M'$).

- each alternating cycle $C$ has an even number of edges

$\Rightarrow$ $C$ has the same number of edges in $M$ and $M'$ (Why?)

- Since $|M'| > |M|$, there must be an odd path $P$ in $H$
  - in which first and last edges belong to $M'$

$\Rightarrow$ $P$ is an $M$-augmenting path, contradiction

$\Rightarrow$ $M$ is a maximum matching $\square$

# Augmenting Paths

Similarly we can prove:

## Corollary

*Let $G = (V, E)$ be a graph, and $M$ and $M^*$ be two matchings in $G$ such that $|M^*| = |M| + k$ for some $k \geq 1$. Then $G$ has at least $k$ pairwise vertex-disjoint $M$-augmenting paths.*

## Proof.

- the subgraph $H = (V, M \otimes M^*)$ consists of alternating paths and cycles with respect to $M$ and $M^*$ (and isolated vertices)
- $M \otimes M^*$ has exactly $k$ more edges from $M^*$ than from $M$

$\Rightarrow$ $H$ contains $k$ pairwise vertex-disjoint odd paths that start and end with edges of $M^*$.

$\Rightarrow$ These paths are $M$-augmenting paths

$\square$

# Augmenting Paths

All the above suggest the following algorithmic method for solving MATCHING:

1. Let $M = \emptyset$
2. Check if there is an $M$-augmenting path $P$
3. If $P$ exists, then $M := M \otimes P$ and go to Step 2. Otherwise output $M$.

- Can we derive a specific algorithm from the above method?
- Where is the catch? At Step 2 !

# Finding augmenting Paths

A reasonable approach:

- for every vertex $v \in V$, search for an $M$-augmenting path starting at $v$ (makes sense only if $v$ is not matched in $M$)
- Definition: a vertex $u$ is $M$-reachable from $v$ if there is an $M$-alternating path from $v$ to $u$

$\Rightarrow$ use a search algorithm to find all $M$-reachable vertices from $v$

- if we find an unmatched $M$-reachable vertex $u$ then: $M$-augmenting path from $v$ to $u$
- otherwise: no $M$-augmenting path from $v$

$\Rightarrow$ try other vertices $v'$

Potential approach for finding $M$-reachable vertices:

- modify a classic search algorithm (BFS, DFS, ...) ?

$\rightarrow$ not so straightforward: problem with odd cycles

$\rightarrow$ but it works for bipartite graphs !

# Finding augmenting Paths

Main idea for the search algorithm:

- grow a search tree rooted at $v$
- each path from $v$ to any vertex $u$ in the tree is alternating

$\Rightarrow$ alternating tree $T$

- vertices that we visit: labeled odd/even (otherwise unlabaled)
- vertex $u$ is odd/even: the path from $v$ in $T$ is odd/even
- an unmatched vertex is labeled odd $\Rightarrow$ augmenting path

# Finding augmenting Paths

- the algorithm maintains a LIST of labeled nodes; initially LIST $= \{v\}$, label($v$) = "even"
- iteratively remove a vertex from LIST and continue searching

Examining an even vertex $u$:

- scan its neighbors $N(u)$
- if $w \in N(u)$ is unlabeled: label($w$) = "odd"; add $w$ to LIST
- if $w$ is unmatched $\Rightarrow$ augmenting path found; STOP

Examining an odd vertex $u$ (which is matched):

- explore the unique edge $ux \in M$
- if $x$ is unlabeled: label($x$) = "even"; add $x$ to LIST

The algorithm terminates if:

- either LIST is empty $\Rightarrow$ no augmenting path from $v$,
- or we assign "odd" label to an unmatched vertex $\Rightarrow$ augmenting path

# Finding augmenting Paths

## Definition

A graph $G$ has the unique label property with respect to a matching $M$ and a root vertex $v$, if the search algorithm assigns a unique label (even/odd) to every labeled vertex, regardless of the order in which it visits the vertices.

It is easy to show:

- if a graph has the unique label property, the search algorithm always discovers an augmenting path (if it exists)

Consider an augmenting path $P = (v, x_1, y_1, \ldots, x_k, y_k, u)$:

- if the algorithm visits vertices in this order

$\Rightarrow$ $u$ becomes "odd"

- the same happens by following any other order !

Some graphs possess this property:

- among them also bipartite graphs

# Maximum Matchings in Bipartite Graphs

Recall that a graph is bipartite if its vertex set can be partitioned into two (possibly empty) sets:

- $R$ (red)
- $B$ (blue)

such that every edge has one end-vertex in $R$ and the other one in $B$.

$\rightarrow$ we can safely assume that $|R| \leq |B| \;\Rightarrow\; |R| \leq \frac{n}{2}$

We describe how to perform Step 2 for bipartite graphs in the MATCHING algorithm from the previous slide:

1. Let $M = \emptyset$
2. Check if there is an $M$-augmenting path $P$
3. If $P$ exists, then $M := M \otimes P$ and go to Step 2. Otherwise output $M$.

# Alternating trees in bipartite graphs

Let $M$ be a matching in a bipartite graph $G = (R \cup B, E)$.
If all red vertices are matched, then $M$ is maximum.
Otherwise we try to find an $M$-augmenting path $P$ (if it exists):

1. Start with an unmatched red vertex $v$.
2. Consider all blue neighbors of $v$.
   If one of them is unmatched, STOP (we found $P$ ! ).
3. Consider the unique matched red neighbor of every newly found blue vertex.
4. Consider all new blue neighbors of the red vertices of Step 3. If one of them is unmatched, STOP (we found $P$ ! ).
   If no new blue neighbor is detected, STOP.
   Otherwise go to Step 3.

If the algorithm terminated because no new blue neighbour was found in Step 4, then we found an alternating tree $T$.

Consider the (bipartite) graph $G - T$ and repeat the process until an augmenting path $P$ is found or the set of unmatched red vertices has become empty.

# Correctness

Suppose no augmenting path (with respect to $M$) is found in Steps 2 or 4.

So we have found an alternating tree $T$. By construction, $v$ is not an end-vertex of an augmenting path $P$.

We claim that no other vertex in $T$ is used for any such augmenting path either.

1. Observe that there are no edges from (the final) red vertices to (blue) vertices outside $T$ (as our algorithm has terminated).
2. Any edge from a blue vertex in $T$ to a red vertex outside $T$ cannot be on any other augmenting path either, otherwise we can modify $P$ into a new $M$-augmenting path $P'$ starting at $v$.

$\Rightarrow$ we can safely remove the vertices of $T$ and consider $G - T$.

# Running Time

Since we only explore each edge once, the augmenting path algorithm runs in $O(|E|)$ time.

Each occasion results in a matching that has one more edge than the previous matching. Hence we run the augmenting path algorithm at most $\frac{|V|}{2}$ times (only from red vertices).

Conclusion: The total running time of the algorithm for finding a maximum matching in a bipartite graph is $O(|E||V|)$ time.

Hopcroft-Karp algorithm (1973) improves to $O(|E|\sqrt{|V|})$ time.

Mucha and Sankowski (2004) used fast matrix multiplication to get a randomized algorithm with time $O(|V|^{2.376})$.

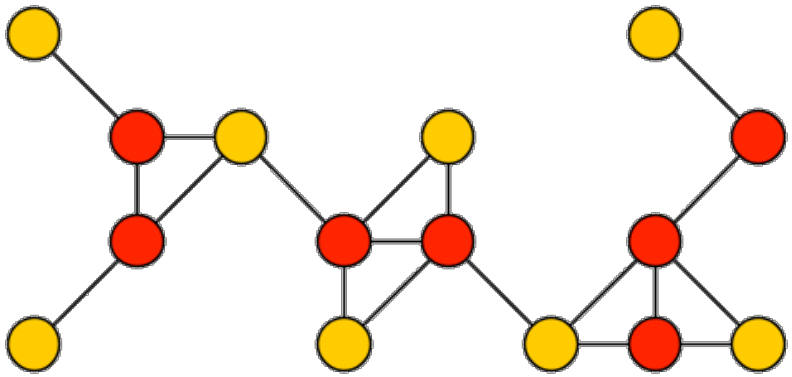
General Graphs. Edmond's algorithm (1965) for MATCHING, based on the bipartite algorithm, takes $O(|E||V|^2)$ time.

Micali & Vazirani (1980) improved to $O(|E|\sqrt{|V|})$ time (fastest).

$\Rightarrow$ same complexity as bipartite graphs

# Connections To Other Problems: Vertex Cover

Suppose you have only a small number of cameras which you want to place at road junctions (vertices) in a city so that every road is "covered".



A vertex cover of a graph $G = (V, E)$ is a set $S \subseteq V$ so that each edge of $G$ has at least one end-vertex in $S$.
The vertex cover number $\tau_G$ of $G$ is the size of a smallest vertex cover of $G$.

| | |
|---|---|
| VERTEX COVER | |
| *Instance:* | a graph $G$. |
| *Task:* | determine $\tau_G$. |

# Vertex Cover for Bipartite Graphs

Computing the vertex cover number of a graph is computationally hard (NP-hard).

However, this may change if we restrict the input to some class of graphs.

Question: what about bipartite graphs?

**Theorem**

*Let G be a bipartite graph. Then* $\tau_G = \nu_G$.

$\Rightarrow$ we can use the matching algorithm for bipartite graphs to compute $\tau_G$ as well !

# Proof that $\tau_G = \nu_G$ for Bipartite Graphs $G$

Let $M$ be a matching and $C$ be a vertex cover of $G \Rightarrow$ at least one end-vertex of every edge in $M$ belongs to $C$, $\Rightarrow |M| \leq |C|$

Let $M'$ be a maximum matching of $G$. We will build a vertex cover $C'$ with $|M'| = |C'|$, so $C'$ has minimum size.

As $M'$ has maximum size, the matching algorithm for bipartite graphs constructs a forest $F$ of disjoint alternating trees (always rooted at red vertices).

Let $M_1$ be the set of edges of $M'$ in $F$. Let $M_2 = M' \setminus M_1$.

Then $C'$ has all blue vertices in $M_1$ and all red vertices in $M_2$.

$\Rightarrow$ $C'$ has the blue labeled and the red unlabeled vertices

Then $|C'| = |M_1| + |M_2| = |M'|$. We will also show that $C'$ is indeed a vertex cover $\Rightarrow$ $C'$ is a minimum vertex cover.

# Proof that $\tau_G = \nu_G$ for Bipartite Graphs $G$

Reminder:

- forest $F$ of alternating trees (always rooted at red vertices)
- $M_1$ is the set of edges of $M$ in $F$; $M_2 = M \setminus M_1$
- $C'$ has blue labeled ($M_1$) and red unlabeled ($M_2$) vertices

Proof by contradiction:

- suppose that there is an edge $vu$ such that $v, u \notin C'$ (where $v$ is red and $u$ is blue)
- if $vu \in M = M_1 \cup M_2$ then:
  - if $vu \in M_1$ then $u \in M_1 \subseteq C'$, contradiction
  - if $vu \in M_2$ then $v \in M_2 \subseteq C'$, contradiction
- let $vu \notin M$:
  - note that $v \notin M_2$ and $u \notin M_1$, otherwise contradiction

  $\Rightarrow$ $v$ is in some alternating tree in $F$ (i.e. $v$ is labeled "even"), and $u$ is not in any alternating tree (i.e. $u$ is unlabeled)

  $\Rightarrow$ our algorithm would detect the blue neighbor $u$ of the labeled red vertex $v$ $\Rightarrow$ $u \in M_1$, contradiction

$\Rightarrow$ $C'$ is a vertex cover

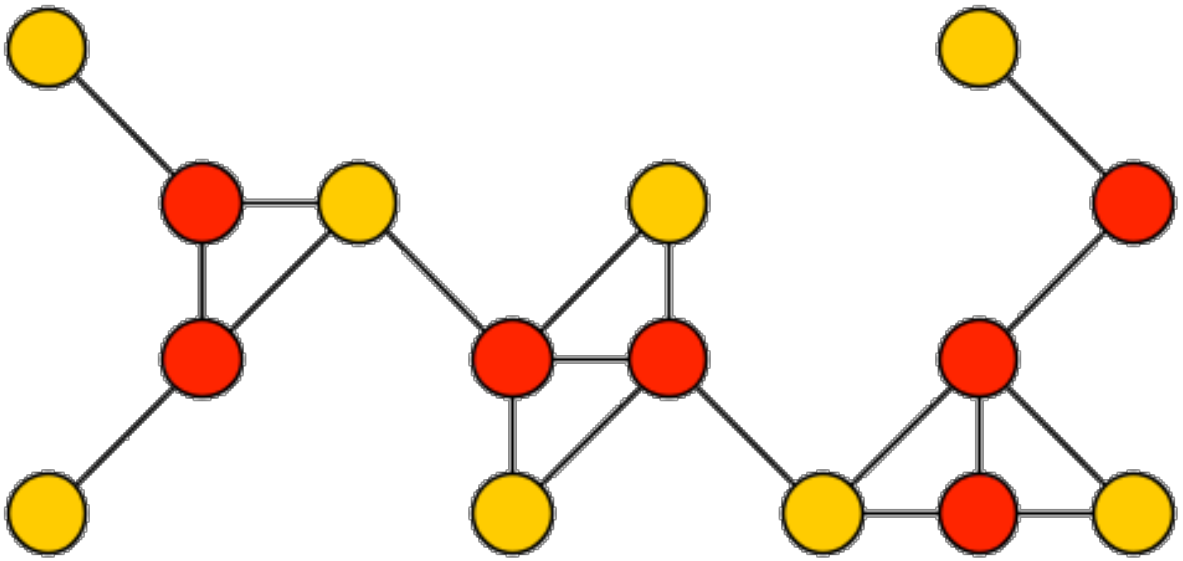
# Vertex Cover for General Graphs

| VERTEX COVER | |
|---|---|
| *Instance:* | a graph $G$ and integer $k \geq 0$. |
| *Task:* | does $G$ have a vertex cover of size at most $k$? |

- Recall that this problem is computationally hard.
- Let $n$ and $m$ be the number of vertices and edges of $G$.
  Brute force requires checking $O(n^k)$ possible choices for $S$.
- Now assume $k$ is relatively small compared to $n$

  Can we find an algorithm that runs in $f(k) + O(n + m)$ time?

# Reduction Rules



**Rule 1.** *a road junction (vertex) covers at least one road:*
If vertex $v$ is not an end-vertex of an edge then we don't need to install a camera
$\Rightarrow$ remove $v$, so consider the smaller instance $(G - v, k)$.

**Rule 2.** *a road junction covering more than $k$ roads must be used:*
If $v$ has at least $k + 1$ neighbors, then put $v$ in the vertex cover (otherwise we need too many cameras!) and consider the smaller instance $(G - v, k - 1)$.

**Rule 1.** If $v$ is not an end-vertex of an edge, then remove $v$, so consider $(G - v, k)$.

**Rule 2.** If $v$ has at least $k + 1$ neighbors, then put $v$ in the vertex cover and consider $(G - v, k - 1)$.

At some point, Rules 1 and 2 can not be applied anymore. Let $(G', k')$ be the resulting instance, where $k' \leq k$. Note that we computed $(G', k')$ in $O(n + m)$ time.

- If $G'$ has at most $k + k^2$ vertices: We solve the problem on $G'$ using brute force. This takes $f(k)$ time for some function $f$ that only depends on $k$ $\Rightarrow$ running time $f(k) + O(m + n)$
- If $G'$ has more than $k + k^2$ vertices, then the answer is **NO**: For contradiction, assume $S$ is vertex cover of $G'$ with $|S| \leq k' \leq k$. By Rule 2, there are at most $k|S| \leq k^2$ vertices which are not in $S$ but have a neighbor in $S$.

$\Rightarrow$ there exists a vertex $u$ in $G'$ which is not in $S$ and has no neighbor $S$. By Rule 1, $u$ has a neighbor $v$, so $u$ or $v$ must be in $S$ (since $S$ is a vertex cover), a contradiction.